Giving Every Robot a Unique ID

Three-phase ID assignment on Blinky Blocks (VisibleSim)

Muhammad Omais Rana
Master IoT, University of Franche-Comté

November 5, 2026

Abstract

In a modular robot, every module (particle) needs a unique ID to send messages, route them and match two configurations, but modules are tiny, have very little memory and energy, and no central computer exists. We make the modules give themselves unique IDs by passing short messages to the neighbours they touch. The algorithm has three phases: build a tree from a leader, count the modules in every branch, then hand out disjoint ranges of IDs along the tree. All IDs have the same, shortest possible length $\lceil \log_2 n \rceil$ bits. We implemented it in VisibleSim and tested it on four structures (a diamond, a maze, random islands joined by chains, and a spiral), checking uniqueness, number of messages and real execution time.

1 The problem in one picture

Each module is a small cube that knows only which of its borders touch another module and can talk to those neighbours. The shape of the whole robot changes over time, so a module needs a name that never changes.

Goal: give each of the n modules a different ID, as short as possible, using only messages between neighbours.

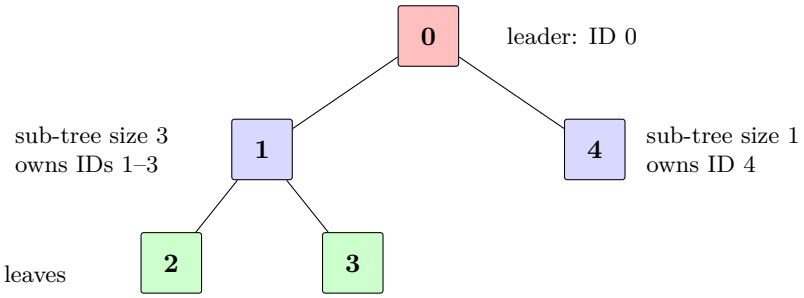


Figure 1: A 5-module tree. The leader takes ID 0 and gives each child a block of IDs as large as its sub-tree. Module 1 owns $\{1, 2, 3\}$, module 4 owns $\{4\}$. Blocks never overlap, so IDs are unique without any negotiation. $n = 5$ needs only $\lceil \log_2 5 \rceil = 3$ bits.

Why not simply write an ID at the factory, or let each module choose a random one? A factory ID must be very long to be unique worldwide and needs extra manufacturing steps. Random IDs need to be very long to make two equal choices unlikely. Long IDs cost memory and energy because every message carries them. Section **??** compares the options.

2 How the modules find their IDs

All modules run the same program. A leader has been elected beforehand and can be anywhere. Every step is just modules passing short messages to a neighbour. Table 1 lists the five message types.

Type	Name	Direction	Meaning	Data
1	EXPLORE	module → neighbours	"Do you want to be my child?"	–
2	CONFIRM	module → parent	"Yes, I am now your child."	–
3	DECLINE	module → sender of 1	"No, I already have a parent."	–
4	REPORT	child → parent	"My sub-tree contains s modules."	s
5	ASSIGN	parent → child	"Your ID is x."	x

Table 1: The five message types.

1. **Explore (build a tree).** The leader marks itself "discovered" and sends EXPLORE to all its neighbours. A module that receives its first EXPLORE remembers the sender as its *parent*, answers CONFIRM, and sends EXPLORE to all its other neighbours (never back to the parent). A module that is already discovered answers DECLINE. Whoever explores a module first becomes its parent, so a tree rooted at the leader appears, with no temporary IDs needed.

2. **Count (report sizes).** This runs at the same time as step 1. Each module counts how many of its EXPLOREs are still unanswered. When all are answered and all its children have reported, its sub-tree is complete and it sends REPORT with its *sub-tree size* (itself plus the sizes of its children) to its parent. A leaf reports 1. The sizes flow from the leaves to the leader, which then knows the total n and computes the ID length $\lceil \log_2 n \rceil$ bits.

3. **Assign (hand out IDs).** The leader takes ID 0. It sends ASSIGN to its children: the first child gets ID + 1, and the i-th child gets

$$\mathrm{ID} + 1 + \sum_{k=1}^{i-1} S_k,$$

   where S_k is the sub-tree size of the k-th child. A module that receives ID x with sub-tree size s owns the IDs $x, \ldots, x+s-1$ and splits $x+1, \ldots, x+s-1$ among its own children the same way.

Result. The n modules hold exactly the IDs $0, \ldots, n-1$, each coded on the same number of bits.

3 Implementation in VisibleSim

We used Blinky Blocks, placed flat in one plane ($z = 0$) so that each module touches at most four neighbours, like the square modules of the paper. The code has three files in the same layout as the other VisibleSim block codes.

- `idAssign.cpp` is the main program; `idAssignCode.hpp/.cpp` is the block code with one handler per message type (`onExplore`, `onConfirm`, `onDecline`, `onReport`, `onAssign`).

- **Leader.** In the XML a block with `isLeader="true"` is the leader. For the random choice every block carries `randomLeader="true"`, and one block is drawn at random at the start of each run.
- **Checking.** Simulator-side counters (not used by the algorithm) record the messages sent per type, the set of IDs handed out, and the time. When the last ID is assigned, a report is printed and shown in the VisibleSim info window.
- **Time.** We measure *real* (wall-clock) time with `std::chrono::steady_clock`, from the leader's start to the last ID assigned, and also from the start to the moment the leader knows n (end of phases 1+2).
- **Colours.** The leader is white, the last module to receive its ID is black, and every other module takes a colour computed from its ID when it receives it, so the third phase can be watched spreading outward.

4 Tests

We tested four structures, all flat. They differ in the number of links between modules, which is what the number of messages depends on, and in how deep the tree becomes.

Structure	Description	n	Links E	Loops	ID bits	Leader
diamond	filled rhombus, many loops	145	256	112	8	fixed, centre
maze	single-block corridors, no loops	125	124	0	7	random each run
random	7 compact islands joined by chains	200	280	81	8	random each run
spiral	rectangular spiral, one chain	150	149	0	8	random each run

Table 2: Structures tested. E is the number of pairs of touching modules; loops $= E - (n - 1)$.

4.1 Results

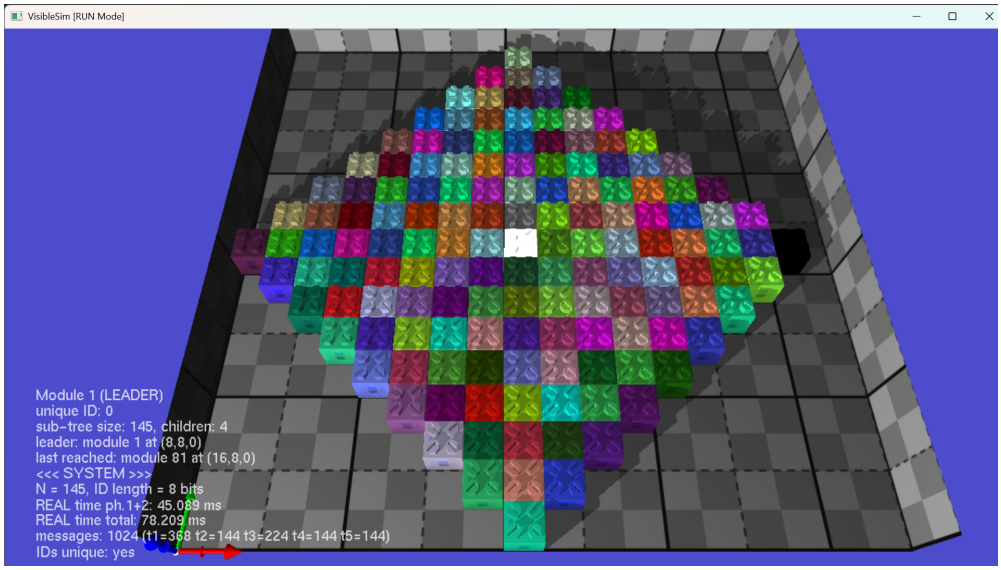
Table 3 gives one representative run per structure. For the three random-leader structures, Table 4 shows several runs with different leaders.

Structure	Leader	Real time (ms)		Messages						$4E$	IDs OK
	module	ph. 1+2	total	t1	t2	t3	t4	t5	total	expected	
diamond	1	44.260	72.514	368	144	224	144	144	1024	1024	Yes
maze	60	356.817	514.022	124	124	0	124	124	496	496	Yes
random	90	171.179	263.996	361	199	162	199	199	1120	1120	Yes
spiral	148	555.803	836.950	149	149	0	149	149	596	596	Yes

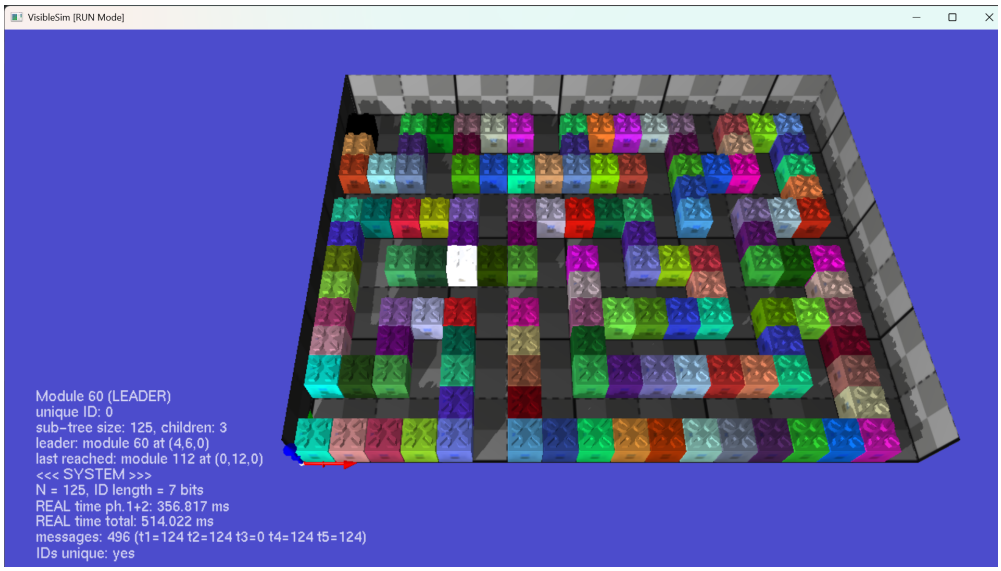
Table 3: Measured results, one run per structure. The expected number of messages is $4E$ (see Section 5).

4.2 What we observe

- **IDs are unique and complete.** In all 3 runs the "IDs cover $0..n-1$" with no duplicates.
- **ID length.** The IDs used 8 / 7 / 8 / 8 bits for the four structures, the minimum $\lceil \log_2 n \rceil$.
- **Messages follow the number of links.** The measured total matched $4E$ in all runs.
- **Loops cost decline messages.** Type 3 messages appear only where loops exist.

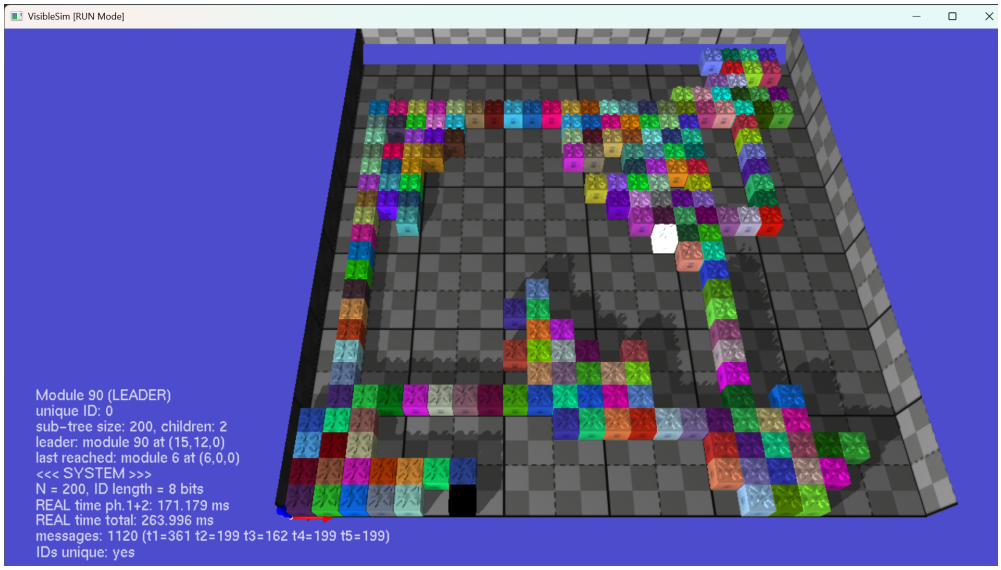


(a) diamond, 145 modules

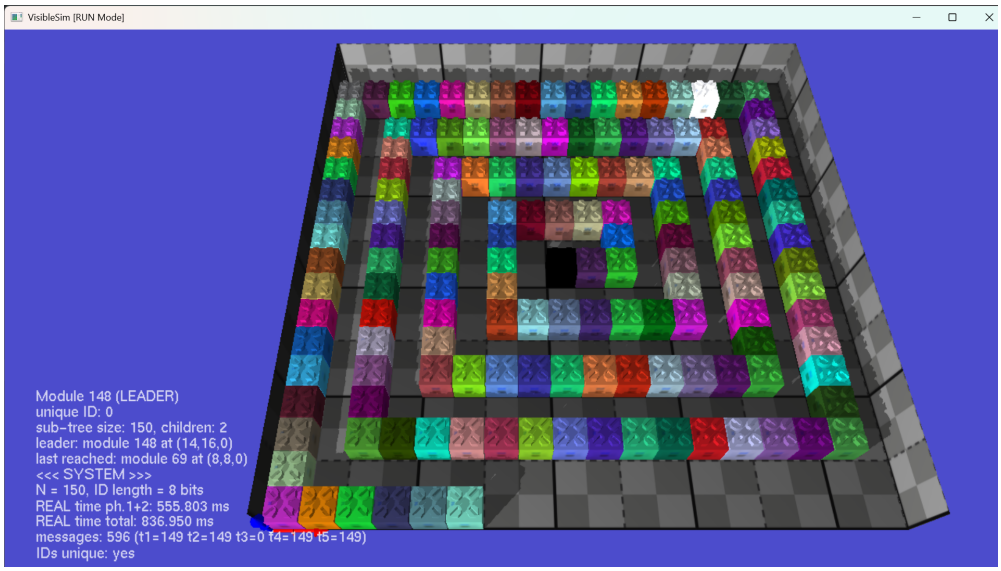


(b) maze, 125 modules

Figure 2: The diamond and the maze at the end of a run. White = leader, black = last module reached, other colours come from each module's ID.



(a) random islands and chains, 200 modules



(b) spiral, 150 modules

Figure 3: The random islands and the spiral at the end of a run. White = leader, black = last module reached, other colours come from each module's ID.

Structure	Run	Leader module	Last module reached	Total time (ms)
maze	1	103	1	525.623
	2	29	1	372.287
	3	81	112	482.242
random	1	90	6	263.984
	2	69	200	278.984
	3	61	200	264.352
spiral	1	110	6	921.211
	2	120	6	590.213
	3	108	6	899.443

Table 4: Several runs per structure with a randomly drawn leader.

5 Complexity

We use: n = number of modules; E = number of pairs of touching modules; D = depth of the tree built in phase 1 (the longest chain of parents from a module to the leader, $D \leq n - 1$).

Complexity		In plain words
Communication	exactly $4E$ messages; $O(n)$	Every link between two touching modules carries exactly 4 messages. On a link of the tree: explore, confirm, report, assign. On any other link: an explore and a decline in each direction. With at most 4 neighbours, $E \leq 2n$, so at most $8n$ messages; for loop-free shapes (maze, spiral, chains) $E = n - 1$.
Time (parallel)	about $3D$ message delays; $O(D)$	Phases 1 and 2 take one trip down the tree and one back up, phase 3 one more trip down. A short tree is fast, a long chain is slow: D is about n for a chain with the leader at its end, and about half of that if the leader is in the middle.
Memory per module	$O(1)$	A parent, at most 4 children with a size each, three counters, and the final ID of $\lceil \log_2 n \rceil$ bits.
ID length	$\lceil \log_2 n \rceil$ bits	The smallest length that gives n different IDs.

Table 5: Complexity of the three-phase ID assignment.

6 Conclusion

Modules that only talk to their neighbours can give themselves unique IDs of minimal length: they build a tree from the leader, count the modules in each branch, and hand out disjoint blocks of IDs. The number of messages is exactly four per link, and the parallel time follows the depth of the tree, so shape and leader position matter for time but not for the message count of loop-free shapes.